

01_main-py

#python

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ main.py
```

El archivo `main.py` se encuentra dentro de la carpeta `backend/app/`.

Este archivo es el **punto de entrada principal del backend FastAPI**. Es decir, es el archivo que Uvicorn carga cuando se ejecuta la API desde Docker Compose.

En el archivo `docker/compose.yml`, la API se arranca con el siguiente comando:

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

La parte importante es:

```
app.main:app
```

Esto significa:

<code>app</code>	→ carpeta Python <code>backend/app/</code>
<code>main</code>	→ archivo <code>main.py</code>
<code>app</code>	→ variable <code>app</code> definida dentro de <code>main.py</code>

Por tanto, este archivo es el núcleo inicial desde el que se crea la aplicación FastAPI y se conectan las rutas del laboratorio.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,260p' backend/app/main.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,260p'
```

Indica que se impriman las líneas de la 1 a la 260.

```
backend/app/main.py
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
# backend/app/main.py

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.api.routes.info import router as info_router
from app.api.routes.alerts import router as alerts_router
from app.api.routes.events import router as events_router
from app.api.routes.health import router as health_router
from app.api.routes.ingest import router as ingest_router
from app.api.routes.metrics import router as metrics_router
from app.api.routes.rules import router as rules_router

app = FastAPI(title="SIEM Backend", version="0.1.0")

# CORS (DEV): permite el frontend servido localmente.
# Ajusta el/los orígenes a tu puerto real (p.ej. 5173 si usas Vite).
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Routers
app.include_router(health_router)
app.include_router(events_router)
app.include_router(ingest_router)
app.include_router(rules_router)
app.include_router(alerts_router)
app.include_router(info_router)
app.include_router(metrics_router)
```

4 Función general del archivo

El archivo `main.py` cumple varias funciones importantes dentro del backend:

1. Importa FastAPI.
2. Importa el middleware CORS.
3. Importa los routers definidos en otros archivos.
4. Crea la instancia principal de la aplicación.
5. Configura CORS para permitir peticiones desde el frontend local.
6. Registra los routers de la API.

Este archivo no contiene directamente la lógica de eventos, reglas o alertas. Su función principal es **ensamblar la aplicación**.

La lógica concreta está separada en archivos dentro de:

```
backend/app/api/routes/
```

Por tanto, `main.py` actúa como un archivo de arranque y organización.

5 Estructura general del archivo

El archivo puede dividirse en cuatro bloques:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
```

Importaciones principales de FastAPI y CORS.

```
from app.api.routes.info import router as info_router
from app.api.routes.alerts import router as alerts_router
from app.api.routes.events import router as events_router
from app.api.routes.health import router as health_router
from app.api.routes.ingest import router as ingest_router
from app.api.routes.metrics import router as metrics_router
from app.api.routes.rules import router as rules_router
```

Importación de routers.

```
app = FastAPI(title="SIEM Backend", version="0.1.0")
```

Creación de la aplicación FastAPI.

```
app.add_middleware(...)
```

Configuración de CORS.

```
app.include_router(...)
```

Registro de rutas en la aplicación principal.

Visualmente:

```
main.py
├─ Importaciones de FastAPI
├─ Importaciones de routers
├─ Creación de app
├─ Configuración CORS
└─ Inclusión de routers
```

6 Análisis línea por línea

Comentario inicial

```
# backend/app/main.py
```

Esta línea es un comentario.

En Python, todo lo que empieza por `#` se considera comentario y no se ejecuta.

Aquí sirve para indicar la ruta del archivo dentro del proyecto:

```
backend/app/main.py
```

No afecta al funcionamiento del backend. Es una referencia visual para el desarrollador.

Importación de FastAPI

```
from fastapi import FastAPI
```

Esta línea importa la clase `FastAPI` desde el paquete `fastapi`.

Desglose:

```
from fastapi
```

Indica que se va a importar algo desde la librería `fastapi`.

```
import FastAPI
```

Importa la clase `FastAPI`.

La clase `FastAPI` se utiliza para crear la aplicación principal.

Más adelante se usa aquí:

```
app = FastAPI(title="SIEM Backend", version="0.1.0")
```

Sin esta importación, Python no sabría qué es `FastAPI` y aparecería un error como:

```
NameError: name 'FastAPI' is not defined
```

o, si la librería no estuviera instalada:

```
ModuleNotFoundError: No module named 'fastapi'
```

Importación del middleware CORS

```
from fastapi.middleware.cors import CORSMiddleware
```

Esta línea importa `CORSMiddleware`.

Desglose:

```
from fastapi.middleware.cors
```

Indica que el elemento se importa desde el módulo de CORS incluido en FastAPI/Starlette.

```
import CORSMiddleware
```

Importa el middleware que permite configurar CORS.

CORS significa:

```
Cross-Origin Resource Sharing
```

Es un mecanismo de seguridad del navegador que controla si una página web cargada desde un origen puede hacer peticiones a otro origen diferente.

Por ejemplo:

```
Frontend: http://localhost:5173  
Backend:  http://localhost:8000
```

Aunque ambos están en `localhost`, los puertos son distintos. Para el navegador, eso ya cuenta como un origen diferente.

Por eso se configura CORS en este archivo.

Importación del router `info`

```
from app.api.routes.info import router as info_router
```

Esta línea importa el objeto `router` desde el archivo:

```
backend/app/api/routes/info.py
```

Pero lo renombra como:

```
info_router
```

Desglose:

```
from app.api.routes.info
```

Ruta de importación dentro del proyecto.

Se interpreta así:

```
app      → carpeta backend/app/  
api      → subcarpeta backend/app/api/  
routes   → subcarpeta backend/app/api/routes/  
info     → archivo info.py
```

```
import router
```

Importa el objeto llamado `router` definido dentro de `info.py`.

```
as info_router
```

Renombra ese objeto para que en `main.py` tenga un nombre más descriptivo.

Esto es útil porque todos los archivos de rutas suelen definir un objeto llamado `router`. Si se importaran todos con el mismo nombre, se pisarían entre sí.

Por tanto:

```
router as info_router
```

significa:

```
Importa el router de info.py, pero en este archivo llámalo info_router.
```

Importación del router `alerts`

```
from app.api.routes.alerts import router as alerts_router
```

Importa el router definido en:

```
backend/app/api/routes/alerts.py
```

y lo renombra como:

```
alerts_router
```

Este router agrupa las rutas relacionadas con la gestión de alertas.

Normalmente contiene endpoints para consultar alertas, filtrar alertas o actualizar su estado.

La idea es que `main.py` no tenga que definir directamente las rutas de alertas. Solo importa el router y lo conecta a la aplicación principal.

Importación del router `events`

```
from app.api.routes.events import router as events_router
```

Importa el router definido en:

```
backend/app/api/routes/events.py
```

y lo renombra como:

```
events_router
```

Este router agrupa las rutas relacionadas con los eventos almacenados.

Los eventos representan la información de seguridad que entra en el laboratorio SIEM y que después puede ser evaluada por reglas.

Importación del router `health`

```
from app.api.routes.health import router as health_router
```

Importa el router definido en:

```
backend/app/api/routes/health.py
```

y lo renombra como:

```
health_router
```

Este router suele contener un endpoint sencillo para comprobar que la API está viva.

Por ejemplo:

```
GET /health
```

Este tipo de endpoint es útil para pruebas, monitorización y comprobación rápida del servicio.

Importación del router `ingest`

```
from app.api.routes.ingest import router as ingest_router
```

Importa el router definido en:

```
backend/app/api/routes/ingest.py
```

y lo renombra como:

```
ingest_router
```

Este router es clave en el laboratorio, porque representa la entrada de eventos al sistema.

La ingesta es el proceso mediante el cual el SIEM recibe eventos de seguridad para analizarlos posteriormente.

Importación del router `metrics`

```
from app.api.routes.metrics import router as metrics_router
```

Importa el router definido en:

```
backend/app/api/routes/metrics.py
```

y lo renombra como:

```
metrics_router
```

Este router agrupa rutas relacionadas con métricas o resúmenes del sistema.

Puede servir para obtener datos agregados, como número de eventos, número de alertas o distribución por severidad, dependiendo de la implementación concreta.

Importación del router `rules`

```
from app.api.routes.rules import router as rules_router
```

Importa el router definido en:

```
backend/app/api/routes/rules.py
```

y lo renombra como:

```
rules_router
```

Este router agrupa las rutas relacionadas con reglas de detección.

Las reglas son condiciones configurables que el sistema utiliza para decidir si un evento debe generar una alerta.

Creación de la aplicación FastAPI

```
app = FastAPI(title="SIEM Backend", version="0.1.0")
```

Esta línea crea la instancia principal de FastAPI.

Desglose:

```
app
```

Es el nombre de la variable donde se guarda la aplicación.

Este nombre es muy importante porque Uvicorn lo busca cuando ejecuta:

```
uvicorn app.main:app
```

La última parte de `app.main:app` hace referencia precisamente a esta variable.

```
=
```

Asigna a la variable `app` el resultado de crear una instancia de `FastAPI`.

```
FastAPI(...)
```

Llama al constructor de la clase `FastAPI`.

```
title="SIEM Backend"
```

Define el título de la API.

Este título aparece en la documentación automática de Swagger, normalmente en:

```
http://localhost:8000/docs
```

```
version="0.1.0"
```

Define la versión de la API.

También aparece en la documentación generada automáticamente.

Esta línea es una de las más importantes del archivo, porque crea el objeto central de la aplicación.

Sin esta variable `app`, Uvicorn no podría arrancar la API con `app.main:app`.

Comentario sobre CORS

```
# CORS (DEV): permite el frontend servido localmente.
```

Comentario explicativo.

Indica que la configuración CORS está pensada para desarrollo.

La palabra `DEV` hace referencia a entorno de desarrollo.

El comentario explica que CORS permite que el frontend servido localmente pueda hacer peticiones al backend.

```
# Ajusta el/los orígenes a tu puerto real (p.ej. 5173 si usas Vite).
```

Segundo comentario relacionado con CORS.

Explica que se deben ajustar los orígenes permitidos según el puerto real donde se esté sirviendo el frontend.

Por ejemplo, Vite suele usar:

```
http://localhost:5173
```

En este proyecto se ha dejado ese origen como permitido.

Añadir middleware CORS

```
app.add_middleware(
```

Esta línea llama al método `add_middleware` de la aplicación FastAPI.

Desglose:

```
app
```

Es la instancia principal de FastAPI creada anteriormente.

```
.add_middleware
```

Es un método que permite añadir middleware a la aplicación.

Un middleware es una capa intermedia que procesa las peticiones o respuestas antes o después de llegar a los endpoints.

```
(
```

Abre la llamada a la función. Los argumentos se escriben en varias líneas para mejorar la legibilidad.

```
CORSMiddleware,
```

Primer argumento de `add_middleware`.

Indica qué tipo de middleware se quiere añadir.

En este caso se añade:


```
CORSMiddleware
```

que permite configurar qué orígenes pueden llamar a la API desde un navegador.

La coma final indica que hay más argumentos después.

```
allow_origins=["http://localhost:5173"],
```

Este parámetro define qué orígenes pueden hacer peticiones al backend.

Desglose:

```
allow_origins
```

Nombre del parámetro.

```
=
```

Asignación del valor al parámetro.

```
["http://localhost:5173"]
```

Lista de orígenes permitidos.

En Python, los corchetes `[]` definen una lista.

En este caso, la lista contiene un único elemento:

```
http://localhost:5173
```

Esto significa que una aplicación frontend servida desde ese origen podrá llamar a la API sin ser bloqueada por CORS.

Si el frontend estuviera en otro puerto, por ejemplo:

```
http://localhost:3000
```

habría que añadirlo a la lista.

```
allow_credentials=False,
```

Este parámetro indica si se permiten credenciales en las peticiones CORS.

Las credenciales pueden incluir:

```
cookies  
cabeceras de autenticación  
certificados cliente
```

En este proyecto está configurado como:

```
False
```

Esto significa que no se permite el envío de credenciales CORS.

Tiene sentido en un MVP sin autenticación de usuarios.

```
allow_methods=["*"],
```

Este parámetro indica qué métodos HTTP están permitidos en las peticiones CORS.

Desglose:

```
allow_methods
```

Nombre del parámetro.

```
[ "*" ]
```

Lista con el comodín `*`.

El asterisco significa:

```
permitir todos los métodos HTTP
```

Por ejemplo:

```
GET
POST
PUT
PATCH
DELETE
OPTIONS
```

Esto facilita el desarrollo porque no obliga a declarar método por método.

```
allow_headers=[ "*" ],
```

Este parámetro indica qué cabeceras HTTP están permitidas en las peticiones CORS.

El valor:

```
[ "*" ]
```

significa que se permiten todas las cabeceras.

Esto resulta útil en desarrollo, porque evita bloqueos cuando el frontend envía cabeceras como:

```
Content-Type
Authorization
Accept
```

Aunque en un entorno de producción sería recomendable restringirlo más.

```
)
```

Cierra la llamada al método `app.add_middleware`.

Todo este bloque configura CORS para permitir la comunicación entre frontend y backend durante el desarrollo.

Comentario sobre routers

```
# Routers
```

Comentario que marca el comienzo del bloque donde se registran los routers.

No tiene efecto en la ejecución del programa.

Sirve para separar visualmente la configuración CORS de la inclusión de rutas.

Inclusión del router `health`

```
app.include_router(health_router)
```

Esta línea registra el router de salud dentro de la aplicación principal.

Desglose:

```
app
```

Instancia principal de FastAPI.

```
.include_router
```

Método que permite añadir rutas definidas en un `APIRouter`.

```
health_router
```

Router importado desde:

```
backend/app/api/routes/health.py
```

Al incluirlo, los endpoints definidos en `health.py` pasan a formar parte de la API principal.

Inclusión del router `events`

```
app.include_router(events_router)
```

Registra el router de eventos dentro de la aplicación principal.

El router procede de:

```
backend/app/api/routes/events.py
```

Esto hace que los endpoints de consulta de eventos queden disponibles en la API.

Inclusión del router `ingest`

```
app.include_router(ingest_router)
```

Registra el router de ingesta.

El router procede de:

```
backend/app/api/routes/ingest.py
```

Este router permite que la API reciba eventos de seguridad.

Es una de las rutas principales del laboratorio, porque representa la entrada de datos al SIEM.

Inclusión del router `rules`

```
app.include_router(rules_router)
```

Registra el router de reglas.

El router procede de:

```
backend/app/api/routes/rules.py
```

A partir de esta línea, los endpoints relacionados con reglas de detección quedan disponibles en la API.

Inclusión del router `alerts`

```
app.include_router(alerts_router)
```

Registra el router de alertas.

El router procede de:

```
backend/app/api/routes/alerts.py
```

Este router permite consultar o modificar alertas generadas por el sistema.

Inclusión del router `info`

```
app.include_router(info_router)
```

Registra el router de información.

El router procede de:

```
backend/app/api/routes/info.py
```

Normalmente este tipo de router se utiliza para mostrar información general del backend, como versión, estado o metadatos de la aplicación.

Inclusión del router `metrics`

```
app.include_router(metrics_router)
```

Registra el router de métricas.

El router procede de:

```
backend/app/api/routes/metrics.py
```

Este router permite exponer información agregada del sistema.

Puede servir para consultar resúmenes del laboratorio, como volumen de eventos o alertas.

Resultado final del archivo

Después de ejecutar todo el archivo, la aplicación FastAPI queda creada con:

- Configuración CORS.
- Router de salud.
- Router de eventos.
- Router de ingesta.
- Router de reglas.
- Router de alertas.
- Router de información.
- Router de métricas.

Por tanto, cuando Uvicorn arranca `app.main:app`, lo que obtiene es una aplicación FastAPI completa con todos los endpoints registrados.

7 Relación con Docker y Unicorn

Este archivo está directamente conectado con el comando de arranque del contenedor de la API.

En `docker/compose.yml` aparece:

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

La relación es:

```
uvicorn
  ↓
app.main:app
  ↓
backend/app/main.py
  ↓
variable app
  ↓
FastAPI(...)
  ↓
routers incluidos
  ↓
API disponible en localhost:8000
```

Si `main.py` no tuviera la variable `app`, el backend no podría arrancar correctamente.

Si algún import de router fallara, Unicorn tampoco podría cargar la aplicación.

8 Relación con el flujo técnico del laboratorio

`main.py` no procesa eventos directamente, pero conecta las piezas que sí lo hacen.

El flujo relacionado sería:

```
main.py
  ↓
carga ingest_router
  ↓
permite recibir eventos

main.py
  ↓
carga events_router
  ↓
permite consultar eventos

main.py
  ↓
carga rules_router
  ↓
permite gestionar reglas

main.py
  ↓
carga alerts_router
  ↓
permite consultar alertas

main.py
  ↓
carga metrics_router
  ↓
permite consultar métricas
```

Dentro del flujo general del SIEM:

```
Evento de seguridad
↓
Endpoint de ingesta
↓
Validación de datos
↓
Base de datos
↓
Reglas
↓
Alertas
```

`main.py` actúa como el archivo que registra los puntos de entrada para que ese flujo pueda ejecutarse.

9 Errores típicos o puntos importantes

Error al arrancar Uvicorn

Si el comando:

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

falla, puede deberse a que no encuentra el módulo:

```
app.main
```

Causas posibles:

- No se está ejecutando desde el directorio correcto.
- Falta la carpeta backend/app/.
- Falta backend/app/__init__.py.
- El contenedor no tiene montado correctamente el código en /app.

Error porque no existe la variable `app`

Uvicorn busca específicamente:

```
app.main:app
```

La última parte exige que dentro de `main.py` exista una variable llamada:

```
app
```

Si se llamara de otra forma, por ejemplo:

```
api = FastAPI()
```

entonces habría que arrancar Uvicorn con:

```
uvicorn app.main:api
```

En este proyecto se usa correctamente:

```
app = FastAPI(...)
```

Error al importar routers

Si alguna línea como esta falla:

```
from app.api.routes.alerts import router as alerts_router
```

la aplicación no arrancará.

Puede deberse a:

- El archivo alerts.py no existe.
- El archivo no define una variable llamada router.
- Hay un error interno dentro de alerts.py.
- Hay un problema en una dependencia importada por alerts.py.

En FastAPI, un fallo en un import puede impedir que toda la API arranque.

Problemas de CORS

Si el frontend se sirve desde un origen distinto a:

```
http://localhost:5173
```

el navegador puede bloquear las peticiones.

Por ejemplo, si el frontend se abre desde:

```
http://localhost:3000
```

pero `allow_origins` solo contiene:

```
["http://localhost:5173"]
```

entonces habría que añadir el nuevo origen:

```
allow_origins=[  
    "http://localhost:5173",  
    "http://localhost:3000",  
]
```

Uso de comodines en desarrollo

El archivo permite todos los métodos y cabeceras:

```
allow_methods=["*"]  
allow_headers=["*"]
```

Esto es cómodo en desarrollo, pero en producción sería mejor limitar los métodos y cabeceras a los necesarios.

10 Comandos útiles relacionados

Arrancar la API desde Docker Compose:

```
docker compose --env-file docker/.env -f docker/compose.yml up -d api
```

Ver logs del contenedor de la API:

```
docker logs siem-api
```

Ver logs en tiempo real:

```
docker logs -f siem-api
```

Entrar en el contenedor de la API:

```
docker exec -it siem-api bash
```

Comprobar que `main.py` existe dentro del contenedor:

```
docker exec -it siem-api ls -la /app/app/main.py
```

Comprobar que Python puede importar la aplicación:

```
docker exec -it siem-api python -c "from app.main import app; print(app.title)"
```

Comprobar la API desde navegador:

```
http://localhost:8000/docs
```

Comprobar el endpoint de salud:

```
curl http://localhost:8000/health
```

Comprobar OpenAPI JSON:

```
curl http://localhost:8000/openapi.json
```